FixItNow Backend — Project Documentation

Version: 1.0.0
Author: Md Parvej
Date: July 2026

---

1. Product Purpose

FixItNow is a backend REST API for a home services marketplace platform. It connects **customers** who need home services (plumbing, electrical, AC repair, cleaning, etc.) with qualified **technicians** who offer those services.

What the platform does

- Customers browse services and technicians, book appointments, pay online, and leave reviews.
- Technicians create service profiles, set availability, accept/decline bookings, and complete jobs.
- Admins manage users, oversee all bookings, and manage service categories.

Tagline

> *"Your Trusted Home Service Platform"*

---

2. Technology Stack

Layer	Technology
Runtime	Node.js (ESM)
Framework	Express 5
Language	TypeScript
ORM	Prisma 7
Database	PostgreSQL (Neon)
Validation	Zod
Authentication	JWT + bcrypt
Payment Gateways	ShurjoPay, SSLCommerz, Stripe
Deployment	Vercel (Serverless)

---

3. Project Links

Resource	URL
Live API	https://fixitnow-backend-weld.vercel.app
API Base URL	https://fixitnow-backend-weld.vercel.app/api
GitHub Repository	https://github.com/parvejme24/fixitnow_backend

Health Check	https://fixitnow-backend-weld.vercel.app/api/health

Local Development

Resource	URL
Local API	http://localhost:5001/api

---

4. Authentication

Protected routes require JWT Bearer token:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Role	Description
PUBLIC	No token required
CUSTOMER	Customer JWT required
TECHNICIAN	Technician JWT required
ADMIN	Admin JWT required
ANY AUTH	Any valid JWT (Customer, Technician, or Admin)

---

5. Database Tables

Table	Module	Description
`users`	Auth	User accounts, roles, authentication
`technician_profiles`	Technician	Technician bio, skills, rating
`availability_slots`	Technician	Weekly availability per technician
`categories`	Category / Admin	Service categories
`services`	Service / Technician	Services offered by technicians
`bookings`	Booking	Job bookings between customer & technician
`payments`	Payment	Payment transactions
`reviews`	Review	Customer reviews after completed jobs

---

6. Booking Status Flow

```
REQUESTED → ACCEPTED → PAID → IN_PROGRESS → COMPLETED
          ↘ DECLINED
```

```
          ↘ CANCELLED (customer, before IN_PROGRESS)
```

MODULE DOCUMENTATION

Module 1: Health & Root

Related Tables: None (no database for root)

#	Route Name	Method	Route	Access	Behavior
1	Welcome	GET	`/`	Public	Returns welcome message
2	Health Check	GET	`/api/health`	Public	Checks API and database connection

1.1 GET `/`

Access: Public

Request: None

Response (200):

```
Welcome to FixItNow API
```

1.2 GET `/api/health`

Access: Public

Behavior: Verifies database connectivity.

Request: None

Response (200):


```
{
  "success": true,
  "message": "API is healthy",
  "data": {
    "status": "ok",
    "environment": "production",
    "database": "connected",
    "timestamp": "2026-07-07T20:43:03.651Z"
  }
}
```

Module 2: Auth

Related Tables: `users` , `technician_profiles`

#	Route Name	Method	Route	Access	Behavior
1	Register	POST	`/api/auth/register`	Public	Register customer or technician
2	Login	POST	`/api/auth/login`	Public	Login and receive JWT
3	Get Profile	GET	`/api/auth/me`	Any Auth	Get current user profile
4	Update Profile	PUT	`/api/auth/profile`	Any Auth	Update name, phone, password

---

2.1 POST `/api/auth/register`

Access: Public

Behavior: Creates new user. Technicians get auto-created profile.

Request Body:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "password123",
  "phone": "01712345678",
  "role": "CUSTOMER"
}
```

Response (201):

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "user": {
      "id": "cmr85mm0800035najrx5t5wpa",
      "name": "John Doe",
      "email": "john@example.com",
      "phone": "01712345678",
      "role": "CUSTOMER",
      "status": "ACTIVE",
      "createdAt": "2026-07-06T10:00:00.000Z",
      "updatedAt": "2026-07-06T10:00:00.000Z"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

2.2 POST `/api/auth/login`

Access: Public

Behavior: Authenticates user and returns JWT token.

Request Body:

```
{
  "email": "mdparvej@gmail.com",
  "password": "password"
}
```

Response (200):

```
{
  "success": true,
  "message": "Logged in successfully",
  "data": {
    "user": {
      "id": "cmr85mm0800035najrx5t5wpa",
      "name": "Admin",
      "email": "mdparvej@gmail.com",
      "role": "ADMIN",
      "status": "ACTIVE"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

2.3 GET `/api/auth/me`

Access: Any Auth (Bearer token)

Behavior: Returns authenticated user's profile.

Request: None (Bearer token in header)

Response (200):

```
{
  "success": true,
  "message": "User profile fetched successfully",
  "data": {
    "id": "cmr85mm0800035najrx5t5wpa",
    "name": "John Doe",
    "email": "john@example.com",
    "phone": "01712345678",
    "role": "CUSTOMER",
    "status": "ACTIVE",
    "technicianProfile": null
  }
}
```

2.4 PUT `/api/auth/profile`

Access: Any Auth (Bearer token)

Behavior: Updates user name, phone, and/or password.

Request Body:

```
{
  "name": "Updated Name",
  "phone": "01798765432",
  "password": "newpassword123"
}
```

Response (200):

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": {
    "id": "cmr85mm0800035najrx5t5wpa",
    "name": "Updated Name",
    "email": "john@example.com",
    "phone": "01798765432",
    "role": "CUSTOMER",
    "status": "ACTIVE"
  }
}
```

Module 3: Category (Public)

Related Tables: `categories`

#	Route Name	Method	Route	Access	Behavior
1	List Categories	GET	`/api/categories`	Public	Get all service categories

3.1 GET `/api/categories`

Access: Public

Behavior: Returns all service categories.

Request: None

Response (200):

```
{
  "success": true,
  "message": "Categories fetched successfully",
  "data": [
```

```
    {
      "id": "cmr86u1b70001f4aj2pmb0n6n",
      "name": "Plumbing",
      "description": "Pipe and water related services",
      "icon": "🔧",
      "createdAt": "2026-07-05T10:00:00.000Z"
    }
  ]
}
```

Module 4: Service (Public)

Related Tables: `services` , `categories` , `technician_profiles` , `users`

#	Route Name	Method	Route	Access	Behavior
1	List Services	GET	`/api/services`	Public	Get services with filters

4.1 GET `/api/services`

Access: Public

Behavior: Lists active services with pagination and filters.

Query Parameters: `page` , `limit` , `categoryId` , `location` , `minRating` , `minPrice` , `maxPrice` , `search`

Example: `GET /api/services?page=1&limit=10&categoryId=abc&minPrice=500`

Request: None

Response (200):

```
{
  "success": true,
  "message": "Services fetched successfully",
  "meta": { "page": 1, "limit": 10, "total": 5, "totalPages": 1 },
  "data": [
    {
      "id": "cmr8cd3fw00039pajy6j2qabf",
      "title": "Pipe Leak Repair",
      "description": "Fix leaking pipes",
      "price": 1500,
      "isActive": true,
      "category": { "id": "...", "name": "Plumbing", "icon": "🔧" },
      "technician": {
        "id": "cmr87xylm0000gnajs79t2v38",
        "location": "Dhaka",
        "avgRating": 4.5,
        "user": { "id": "...", "name": "Md Parvej" }
      }
    }
```

```
  ]
}
```

Module 5: Technician (Public)

Related Tables: `technician_profiles`, `users`, `services`, `availability_slots`, `reviews`

#	Route Name	Method	Route	Access	Behavior
1	List Technicians	GET	`/api/technicians`	Public	Get all technicians with filters
2	Technician Profile	GET	`/api/technicians/:id`	Public	Get technician details, services, reviews

---

5.1 GET `/api/technicians`

Access: Public

Query Parameters: `page`, `limit`, `location`, `minRating`, `categoryId`, `search`

Response (200):

```
{
  "success": true,
  "message": "Technicians fetched successfully",
  "meta": { "page": 1, "limit": 10, "total": 3, "totalPages": 1 },
  "data": [
    {
      "id": "cmr87xylm0000gnajs79t2v38",
      "bio": "Certified plumber",
      "skills": ["Plumbing", "Pipe fitting"],
      "experienceYears": 5,
      "hourlyRate": 500,
      "location": "Dhaka",
      "avgRating": 4.5,
      "totalReviews": 12,
      "user": { "id": "...", "name": "Md Parvej", "phone": "017..." }
    }
  ]
}
```

5.2 GET `/api/technicians/:id`

Access: Public

Behavior: Full technician profile with services, availability, and reviews.

Response (200):

```
{
  "success": true,
  "message": "Technician profile fetched successfully",
  "data": {
    "id": "cmr87xylm0000gnajs79t2v38",
    "bio": "Certified plumber with 5 years experience",
    "skills": ["Plumbing"],
    "avgRating": 4.5,
    "user": { "name": "Md Parvej", "email": "tech@example.com" },
    "services": [{ "title": "Pipe Leak Repair", "price": 1500 }],
    "availability": [{ "day": "SATURDAY", "startTime": "09:00", "endTime": "18:00"
}],
    "reviews": [{ "rating": 5, "comment": "Great work!" }]
  }
}
```

Module 6: Booking (Customer)

Related Tables: `bookings` , `services` , `technician_profiles` , `users` , `payments` , `reviews`

#	Route Name	Method	Route	Access	Behavior
1	Create Booking	POST	`/api/bookings`	Customer	Create new booking
2	List Bookings	GET	`/api/bookings`	Customer	Get customer's bookings
3	Booking Details	GET	`/api/bookings/:id`	Customer	Get single booking
4	Cancel Booking	PATCH	`/api/bookings/:id`	Customer	Cancel before IN_PROGRESS

6.1 POST `/api/bookings`

Access: Customer

Behavior: Creates booking with status REQUESTED.

Request Body:


```
{
  "serviceId": "cmr8cd3fw00039pajy6j2qabf",
  "scheduledAt": "2026-07-10T10:00:00.000Z",
  "address": "House 12, Road 5, Mirpur 10, Dhaka",
  "notes": "Kitchen sink is leaking"
}
```


Response (201):


```
{
  "success": true,
  "message": "Booking created successfully",
```

```
  "data": {
    "id": "cmr9jqeun0000c8ajp13sb43v",
    "status": "REQUESTED",
    "scheduledAt": "2026-07-10T10:00:00.000Z",
    "address": "House 12, Road 5, Mirpur 10, Dhaka",
    "service": { "title": "Pipe Leak Repair", "price": 1500 },
    "technician": { "user": { "name": "Md Parvej" } }
  }
}
```

6.2 GET `/api/bookings`

Access: Customer

Query: `page` , `limit` , `status`

Response (200):

```
{
  "success": true,
  "message": "Bookings fetched successfully",
  "meta": { "page": 1, "limit": 10, "total": 2, "totalPages": 1 },
  "data": [{ "id": "...", "status": "ACCEPTED", "service": { "title": "..." } }]
}
```

6.3 GET `/api/bookings/:id`

Access: Customer

Response (200):

```
{
  "success": true,
  "message": "Booking details fetched successfully",
  "data": {
    "id": "cmr9jqeun0000c8ajp13sb43v",
    "status": "PAID",
    "payment": { "status": "COMPLETED", "amount": 1500 },
    "review": null
  }
}
```

6.4 PATCH `/api/bookings/:id`

Access: Customer

Behavior: Cancel booking (status must be REQUESTED, ACCEPTED, or PAID).

Request Body:

```
{
  "status": "CANCELLED"
}
```

Response (200):

```
{
  "success": true,
  "message": "Booking cancelled successfully",
  "data": {
    "id": "cmr9jqeun0000c8ajp13sb43v",
    "status": "CANCELLED"
  }
}
```

Module 7: Payment

Related Tables: `payments` , `bookings`

#	Route Name	Method	Route	Access	Behavior
1	ShurjoPay Callback	GET/POST	`/api/payments/shurjopay/callback`	Public	Payment gateway callback
2	Create Payment	POST	`/api/payments/create`	Customer	Start payment session
3	Confirm Payment	POST	`/api/payments/confirm`	Customer	Confirm payment after gateway
4	List Payments	GET	`/api/payments`	Customer	Payment history
5	Payment Details	GET	`/api/payments/:id`	Customer	Single payment details

7.1 POST `/api/payments/create`

Access: Customer

Behavior: Creates payment session. Booking must be ACCEPTED.

Request Body:

```
{
  "bookingId": "cmr9jqeun0000c8ajp13sb43v",
```

```
  "provider": "SHURJOPAY"
}
```

Providers: `SHURJOPAY` , `SSLCOMMERZ` , `STRIPE`

Response (201):

```
{
  "success": true,
  "message": "Payment session created successfully",
  "data": {
    "payment": {
      "id": "cmr9kfsv30000oaajgvhfdrno",
      "transactionId": "SP1783363257829",
      "amount": 1500,
      "status": "PENDING",
      "provider": "SHURJOPAY"
    },
    "gatewayUrl": "https://sandbox.securepay.shurjopayment.com/...",
    "sessionId": null
  }
}
```

7.2 POST `/api/payments/confirm`

Access: Customer

Request Body (ShurjoPay):

```
{
  "paymentId": "cmr9kfsv30000oaajgvhfdrno",
  "orderId": "SP1783363257829"
}
```

Response (200):

```
{
  "success": true,
  "message": "Payment confirmed successfully",
  "data": {
    "id": "cmr9kfsv30000oaajgvhfdrno",
    "status": "COMPLETED",
    "paidAt": "2026-07-06T18:45:00.000Z",
    "booking": { "status": "PAID" }
  }
}
```

7.3 GET `/api/payments`

Access: Customer

Query: `page` , `limit` , `status`

Response (200):

```
{
  "success": true,
  "message": "Payments fetched successfully",
  "meta": { "page": 1, "limit": 10, "total": 1, "totalPages": 1 },
  "data": [{ "id": "...", "status": "COMPLETED", "amount": 1500 }]
}
```

7.4 GET `/api/payments/:id`

Access: Customer

Response (200):

```
{
  "success": true,
  "message": "Payment details fetched successfully",
  "data": {
    "id": "cmr9kfsv30000oaajgvhfdrno",
    "transactionId": "SP1783363257829",
    "status": "COMPLETED",
    "amount": 1500,
    "provider": "SHURJOPAY"
  }
}
```

Module 8: Review (Customer)

Related Tables: `reviews` , `bookings` , `technician_profiles`

#	Route Name	Method	Route	Access	Behavior
1	Create Review	POST	`/api/reviews`	Customer	Review after COMPLETED booking

---

8.1 POST `/api/reviews`

Access: Customer

Behavior: Creates review. Booking must be COMPLETED. Updates technician rating.

Request Body:

```
{
  "bookingId": "cmr9jqeun0000c8ajp13sb43v",
  "rating": 5,
  "comment": "Excellent work, very professional!"
}
```

Response (201):

```
{
  "success": true,
  "message": "Review created successfully",
  "data": {
    "id": "cmr9review001",
    "rating": 5,
    "comment": "Excellent work, very professional!",
    "technician": { "avgRating": 4.6, "totalReviews": 13 }
  }
}
```

Module 9: Technician Management

Related Tables: `technician_profiles`, `availability_slots`, `services`, `bookings`, `categories`

#	Route Name	Method	Route	Access	Behavior
1	Update Profile	PUT	`/api/technician/profile`	Technician	Update bio, skills, location
2	Update Availability	PUT	`/api/technician/availability`	Technician	Set weekly time slots
3	List Services	GET	`/api/technician/services`	Technician	Get own services
4	Create Service	POST	`/api/technician/services`	Technician	Add new service
5	Update Service	PATCH	`/api/technician/services/:id`	Technician	Update service
6	Delete Service	DELETE	`/api/technician/services/:id`	Technician	Delete service
7	List Bookings	GET	`/api/technician/bookings`	Technician	Get assigned bookings
8	Update Booking Status	PATCH	`/api/technician/bookings/:id`	Technician	Accept/decline/complete

---

9.1 PUT `/api/technician/profile`

Request Body:

```
{
  "bio": "Certified plumber with 5 years experience",
  "skills": ["Plumbing", "Pipe fitting"],
  "experienceYears": 5,
  "hourlyRate": 500,
  "location": "Dhaka, Mirpur"
}
```

Response (200):

```
{
  "success": true,
  "message": "Technician profile updated successfully",
  "data": { "id": "...", "bio": "...", "location": "Dhaka, Mirpur" }
}
```

9.2 PUT `/api/technician/availability`

Request Body:

```
{
  "slots": [
    { "day": "SATURDAY", "startTime": "09:00", "endTime": "18:00" },
    { "day": "MONDAY", "startTime": "09:00", "endTime": "18:00" }
  ]
}
```

Response (200):

```
{
  "success": true,
  "message": "Availability updated successfully",
  "data": { "availability": [{ "day": "SATURDAY", "startTime": "09:00", "endTime":
"18:00" }] }
}
```

9.3 POST `/api/technician/services`

Request Body:

```
{
  "title": "Pipe Leak Repair",
  "description": "Fix leaking pipes",
  "price": 1500,
  "categoryId": "cmr86u1b70001f4aj2pmb0n6n"
}
```

Response (201):

```
{
  "success": true,
  "message": "Service created successfully",
  "data": { "id": "...", "title": "Pipe Leak Repair", "price": 1500 }
}
```

9.4 PATCH `/api/technician/bookings/:id`

Request Body (Accept):

```
{ "status": "ACCEPTED" }
```

Allowed transitions:

- REQUESTED → ACCEPTED, DECLINED
- PAID → IN_PROGRESS
- IN_PROGRESS → COMPLETED

Response (200):

```
{
  "success": true,
  "message": "Booking status updated successfully",
  "data": { "id": "...", "status": "ACCEPTED" }
}
```

Module 10: Admin

Related Tables: `users` , `bookings` , `categories` , `services` , `payments`

#	Route Name	Method	Route	Access	Behavior
1	List Users	GET	`/api/admin/users`	Admin	Get all users
2	Update User Status	PATCH	`/api/admin/users/:id`	Admin	Ban/unban user
3	List Bookings	GET	`/api/admin/bookings`	Admin	Get all bookings
4	List Categories	GET	`/api/admin/categories`	Admin	Get categories
5	Create Category	POST	`/api/admin/categories`	Admin	Create category
6	Update Category	PATCH	`/api/admin/categories/:id`	Admin	Update category
7	Delete Category	DELETE	`/api/admin/categories/:id`	Admin	Delete category

10.1 GET `/api/admin/users`

Query: `page` , `limit` , `role` , `status` , `search`

Response (200):

```
{
  "success": true,
  "message": "Users fetched successfully",
  "meta": { "page": 1, "limit": 10, "total": 10, "totalPages": 1 },
  "data": [
    {
      "id": "...",
      "name": "John Doe",
      "email": "john@example.com",
      "role": "CUSTOMER",
      "status": "ACTIVE"
    }
  ]
}
```

10.2 PATCH `/api/admin/users/:id`

Request Body:


```
{ "status": "BANNED" }
```


Response (200):


```
{
  "success": true,
  "message": "User status updated successfully",
  "data": { "id": "...", "status": "BANNED" }
}
```

10.3 POST `/api/admin/categories`

Request Body:


```
{
  "name": "Plumbing",
  "description": "Water and pipe services",
  "icon": "🔧"
}
```


Response (201):


```
{
  "success": true,
  "message": "Category created successfully",
  "data": { "id": "...", "name": "Plumbing", "icon": "🔧" }
}
```


---

7. Standard Response Format

Success:

```
{ "success": true, "message": "...", "data": {} }
```

Error:

```
{ "success": false, "message": "Error description" }
```

---

8. Admin Login Credentials

Field	Value
Email	mdparvej@gmail.com
Password	password

Run `npm run db:seed` to create admin user in database.

---

9. Total API Summary

Module	Routes
Health & Root	2
Auth	4
Category	1
Service	1
Technician (Public)	2
Booking	4
Payment	5
Review	1
Technician Management	8
Admin	7
Total	**35**

FixItNow Backend — Project Documentation v1.0.0